

4 DE MARZO DE 2026



LAYLA: DEFINICIÓN DEL PROBLEMA

DESARROLLO DE SISTEMAS EN RED

ORTIZ GARCIA LUIS DONALDO
FERNÁNDEZ RODRÍGUEZ RODOLFO
UNIVERSIDAD VERACRUZANA
EQUIPO LAYLA

Contenido

Definición del Problema	2
Descripción detallada	2
Contexto actual	2
Consecuencias de no resolverlo	2
Justificación	2
Relevancia del problema	2
Usuarios y roles	4
Estado final ideal	4
Impacto Esperado	5
Técnico	5
Económico / organizacional	5
Académico o social	5
Objetivos de la API	5
Funcionamiento general	5
Motivación	6
Métricas de Éxito	6
Técnicas (Prioridad Fase Académica)	6
De negocio (Prioridad Fase Comercial)	6
De calidad	7
Plan de Trabajo Inicial	7
Actividades principales	7
Entregables parciales	7
Referencias	8

Definición del Problema

Descripción detallada

El ecosistema contemporáneo de la escritura creativa, el diseño de narrativas complejas y la construcción de mundos ficticios (*worldbuilding*) atraviesa una crisis estructural derivada de la profunda fragmentación de las herramientas digitales. Históricamente, la producción literaria y la gestión del canon narrativo han sido tratadas por la industria del software como procesos mutuamente excluyentes, lo que ha obligado a los autores y creadores a depender de una amalgama de plataformas desconectadas para gestionar las múltiples dimensiones de su trabajo intelectual.

Contexto actual

La redacción intensiva de manuscritos suele llevarse a cabo en procesadores de texto tradicionales o software especializado de escritorio de arquitectura monolítica (como Scrivener). Simultáneamente, la construcción de la enciclopedia del mundo se delega a plataformas basadas en la nube o complejas redes de notas (como World Anvil o Obsidian). El panorama competitivo actual ilustra esta dicotomía tecnológica, obligando a los usuarios a comprometer funciones críticas según la herramienta que seleccionen, limitando severamente la coautoría en tiempo real y la portabilidad de los datos.

Consecuencias de no resolverlo

Esta separación artificial entre el texto narrativo y la base de conocimientos genera una fricción operativa masiva. Desde una perspectiva cognitiva, el creador se ve forzado a realizar constantes saltos de contexto (*context switching*). La dependencia de la intervención humana para mantener la coherencia transversal es inherentemente propensa a errores, derivando con frecuencia en inconsistencias narrativas, agujeros de guion y una pérdida sustancial de la productividad. Además, paraliza funcionalmente los esfuerzos modernos de coautoría y edición colaborativa, al no existir un entorno centralizado para el control de versiones preciso y la comunicación intra-equipo.

Justificación

Relevancia del problema

El mercado global de software creativo proyecta un crecimiento masivo, superando los 16.820 millones de dólares en 2026, con el subconjunto de gestión creativa alcanzando los 2.290 millones para 2034. Esto evidencia que el desarrollo de

herramientas para la estructuración narrativa es una necesidad fundamental en las industrias del entretenimiento, autoedición y diseño de videojuegos. Las infraestructuras actuales obligan a un autor moderno a mantener suscripciones activas y concurrentes en múltiples plataformas, fragmentando su ecosistema y elevando sus costos.

Oportunidad identificada

Existe una ausencia crítica de una infraestructura de software integradora, resiliente y distribuida. La principal oportunidad reside en abordar las profundas limitaciones de interoperabilidad mediante la adopción de un modelo de persistencia políglota diseñado específicamente para el dominio narrativo. Al combinar bases de datos documentales para el texto enriquecido y bases de datos orientadas a grafos para el mapeo semántico, Layla se sitúa con una ventaja competitiva frente a soluciones de talla única.

Visión del Sistema

Panorama general

La visión fundacional del ecosistema Layla es consolidarse como la plataforma distribuida y heterogénea definitiva para la creación narrativa. Concibe un ecosistema híbrido compuesto por un Núcleo Transaccional en .NET 10, un Microservicio de Worldbuilding en Node.js y un Middleware de Mensajería con RabbitMQ. La interacción se materializa a través de la estrategia de "Las Tres Pantallas": un Cliente de Escritorio (WPF) *Offline-First* para el trabajo profundo, un Cliente Web (Razor Pages) para administración y lectura pública, y una Aplicación Móvil (Android) como herramienta auxiliar de comunicación por voz.

Diferenciación de Objetivos: Layla 1.0 (Académica) vs. Layla 2.0 (Comercial)

Para comprender la dirección del sistema, es vital separar sus objetivos de acuerdo con su ciclo de vida:

- **Objetivos de la Versión Académica (Layla 1.0 - MVP):** Su propósito fundacional no es la rentabilidad, sino la validación de arquitectura y el aprendizaje. Funciona como un entorno de pruebas (*sandbox*) para demostrar la viabilidad y dominio sobre los sistemas distribuidos. Se diseñó intencionalmente con una alta complejidad técnica (uso simultáneo de C#, Node.js, Kotlin y bases de datos políglotas) para forzar la resolución de problemas de interoperabilidad, consistencia eventual a través de *brokers* de mensajería (RabbitMQ) y resiliencia en red.

- **Objetivos de la Versión Comercial (Layla 2.0 - SaaS):** Su meta es la sostenibilidad económica, la eficiencia operativa y la reducción de la fricción comercial. Busca unificar el ecosistema en el frontend migrando hacia una arquitectura de "Monorepo" (utilizando TypeScript, SvelteKit y Tauri). En cuanto al backend, Layla 2.0 mantiene firmemente .NET 10 como su núcleo tecnológico unificado por su alto rendimiento, con el objetivo de evaluar la migración de los servicios que realizaba Node.js hacia .NET (Minimal APIs) para simplificar la infraestructura de despliegue y abaratar drásticamente el costo de mantenimiento. Asimismo, tiene el objetivo de optimizar la infraestructura de servidores eliminando los WebSockets centralizados de la versión académica en favor del estándar *Peer-to-Peer* (WebRTC) para la comunicación de voz, logrando así un modelo *Freemium* altamente rentable.

Usuarios y roles

El sistema implementa un estricto Control de Acceso Basado en Roles (RBAC):

- **Escritor (Owner):** Nivel de autoridad más alto. Gestiona proyectos, redacta manuscritos y administra permisos. Tiene capacidad bidireccional en sesiones de voz.
- **Editor (Collaborator):** Usuario invitado para co-autoría. Puede modificar contenido y actualizar la wiki, con permisos de audio bidireccional.
- **Lector Registrado:** Consume contenido terminado y navega por grafos. En el módulo de voz, ingresa únicamente como Oyente.
- **Invitado (Guest):** Usuario no autenticado que explora exclusivamente el catálogo público (*Feed*) para incentivar su registro.
- **Administrador:** Gestiona la plataforma, monitorea contenedores y administra usuarios (bloqueos/roles).

Estado final ideal

El estado ideal es un entorno donde la infraestructura técnica sea completamente invisible para el autor, orquestando de manera silenciosa bases de datos complejas, resolución de conflictos espaciales (mediante algoritmos LWW) y flujos de red, garantizando que la única preocupación del usuario sea la calidad de su relato, sin interrupciones por pérdida de conectividad.

Impacto Esperado

Técnico

El proyecto establece un precedente significativo en el diseño de arquitecturas contenerizadas (Docker), manejo de consistencia eventual a través de *brokers* de mensajería (RabbitMQ) y resiliencia adaptativa (*Self-Healing* y patrón *Circuit Breaker*). Demuestra la interoperabilidad entre esquemas transaccionales estrictos (SQL Server), bases de datos documentales masivas (MongoDB) y motores de grafos de alta complejidad relacional (Neo4j).

Económico / organizacional

Layla transforma los flujos de trabajo de los creadores mitigando la sobrecarga cognitiva al colapsar la brecha entre escritura y visualización de entidades. Económicamente, en su visión Layla 2.0, democratiza el acceso a infraestructuras de autoría condensando el ecosistema, reduciendo el Costo Total de Propiedad (TCO) para los escritores. Para la operadora del sistema, la consolidación del backend manteniendo a.NET 10 como tecnología central unificada, junto con la delegación de procesamiento (como WebRTC), habilitan un modelo SaaS altamente escalable y reducen la deuda técnica.

Académico o social

En el contexto de la Licenciatura en Ingeniería de Software de la Universidad Veracruzana, el desarrollo del MVP Layla 1.0 aporta un valor incalculable. Proporciona a los estudiantes un entorno que emula las presiones del mundo real, forzándolos a resolver retos de concurrencia distribuida y persistencia políglota que enfrentan las corporaciones tecnológicas de primer nivel.

Objetivos de la API

Funcionamiento general

La API distribuida en Layla representa la columna vertebral operativa. Es responsable de enrutar comandos y consultas (siguiendo patrones CQRS), actuar como árbitro criptográfico de seguridad mediante validación de tokens JWT sin estado (*stateless*), e interceptar cargas útiles de sincronización determinando jerarquías temporales mediante marcas de tiempo en el servidor (*Server-Side Timestamps*).

Motivación

La motivación primordial es descentralizar la autorización para eliminar la latencia de red transversal y resolver la fricción inherente a mantener sincronizados esquemas documentales y relacionales sin paralizar la interfaz del usuario durante los cálculos de indexación. Además, busca abstraer la complejidad de túneles persistentes para la comunicación bidireccional por voz.

Valor que aporta

- **Consistencia Eventual:** Delega mensajes asíncronos a colas de RabbitMQ, permitiendo que la interfaz del cliente reciba respuestas instantáneas mientras la infraestructura procesa datos pesados en segundo plano.
- **Resolución de Conflictos:** Protege los manuscritos de colisiones de datos resultantes de sincronizaciones diferidas aplicando "La Última Escritura Gana" (LWW).
- **Colaboración Sincrónica:** Establece *Hubs* de *WebSockets* (SignalR) para difundir telemetría de presencia y orquestar el audio *Push-to-Talk* de ultra baja latencia.

Métricas de Éxito

Técnicas (Prioridad Fase Académica)

- **Integridad Operacional:** 100% de respuestas HTTP 200 OK en sondeos de salud post-despliegue en la infraestructura Docker.
- **Tiempo Máximo de Recuperación (RTO):** < 5 minutos para restauración desde catástrofe y recuperación autónoma de contenedores caídos en < 10 segundos.
- **Latencia:** Tiempos de ida y vuelta inferiores a 500 milisegundos en la verificación de identidad *stateless*.
- **Calidad de Código:** Mayor al 70% de cobertura de código en pruebas unitarias e integración continua.

De negocio (Prioridad Fase Comercial)

- **Métricas Financieras:** Crecimiento sostenido del MRR (Ingresos Mensuales Recurrentes) y una Tasa de Retención de Ingresos Netos (NRR) superior al 120%.
- **Adquisición y Optimización:** Optimización del ratio CLTV a CAC (3 a 1) sostenida por la disminución de costos de servidores gracias a la

arquitectura *Peer-to-Peer* (WebRTC), junto con una alta Tasa de Activación Temprana.

De calidad

- **Retención y Abandono:** Reducción del *Churn Rate* a cifras inferiores al 5%, promoviendo mecanismos de retención mediante la propiedad intelectual generada en la plataforma.
- **Satisfacción del Usuario:** Un elevado índice NPS (Net Promoter Score) que valide la fluidez y eficacia del sistema LWW y las sesiones de voz asincrónicas.

Plan de Trabajo Inicial

Actividades principales

El desarrollo del proyecto se divide en seis fases incrementales diseñadas para garantizar la estabilidad de la infraestructura antes de la integración de características colaborativas avanzadas.

Fase	Título de la Fase	Semanas Estimadas	Enfoque Principal
I	Infraestructura	Semanas 1 - 2	Configuración de orquestación y sistemas de seguridad fundamentales.
II	Núcleo (Escritura)	Semanas 3 - 5	Desarrollo del entorno de redacción primario y lógica de persistencia.
III	Distribuido	Semanas 6 - 7	Interconexión de microservicios y bases de datos secundarias.
IV	Clientes Satélite	Semanas 8 - 9	Expansión del ecosistema a plataformas Web y Móvil.
V	Tiempo Real	Semanas 10 - 11	Integración de comunicación sincrónica y transmisión de voz.
VI	Cierre	Semana 12	Aseguramiento de calidad, resiliencia y pruebas finales.

Entregables parciales

Los entregables técnicos mapeados a cada actividad principal son los siguientes:

- **Entregables Fase I:** Despliegue docker-compose operativo con bases de datos y API respondiendo. Sistema funcional de Login/Registro con emisión de JWT.
- **Entregables Fase II:** CRUD de Proyectos conectado a SQL Server. Editor de texto WPF integrado con MongoDB. Modo Offline con caché local y sincronización híbrida básica.
- **Entregables Fase III:** Contenedor Node.js conectado a Neo4j para la creación de nodos de historia. Eventos asíncronos enlazados vía RabbitMQ.
- **Entregables Fase IV:** Portal de Lectura desplegado visualizando el *Feed* público. Aplicación Android nativa con capacidad de consulta y login.
- **Entregables Fase V:** Hub de WebSockets implementado en .NET con pruebas de concurrencia. Módulo *Push-to-Talk* de voz funcional con baja latencia en Android.
- **Entregables Fase VI:** Políticas *restart* en Docker configuradas exitosamente y reportes de pruebas de estrés y *Self-Healing* superadas.

Referencias

- Fortune Business Insights. (2025). *Creative Management Platform Market Size, Share & Industry Analysis*.
- Ortiz García, L. D. (2026). *Layla: Escritura creativa (Documento de proyecto)*. Facultad de Ingeniería de Software, Universidad Veracruzana.
- The Business Research Company. (2025). *Creative Software Global Market Report 2026*.